

From
Beginner
To
Practice

Small
Lab
Big
Future

Linux

Home Lab

HANDBOOK



Build Your Own Linux Environment
for Real DevOps Practice



SET UP
Your Lab
Environment



LEARN
Core Linux
Skills



PRACTICE
Real Commands
and Scenarios



BUILD
Multi-Server
Lab



TROUBLESHOOT
Like a Real
Engineer

- ✓ Step-by-Step Guidance
- ✓ Practical Examples
- ✓ Real-World Scenarios
- ✓ Troubleshooting Labs
- ✓ Beginner Friendly

Practice Today
A Stronger Tomorrow



Servers
Networking
Security
Automation
DevOps

Build
Experiment
Break
Fix
Learn
Repeat

Linux

Home Lab Architecture

Build Your Own Environment. Learn by Doing.

Page 1 of 20

1. WHAT IS A HOME LAB?



A home lab is a personal environment where you create and manage your own servers and infrastructure for learning and practice. It runs on your laptop/PC or on a dedicated machine using virtualization.

2. WHY DEVOPS ENGINEERS BUILD ONE?

- ✓ Gain hands-on Linux experience
- ✓ Practice without real-world risk
- ✓ Learn system administration
- ✓ Test tools and configurations
- ✓ Build skills for DevOps, Cloud and SRE
- ✓ Experiment and make mistakes
- ✓ Prepare for interviews
- ✓ Build a strong technical foundation

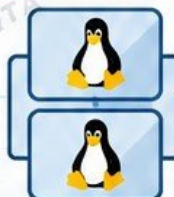
3. LAPTOP/PC → HYPERVISOR → LINUX VMs



Laptop / PC
Your physical machine

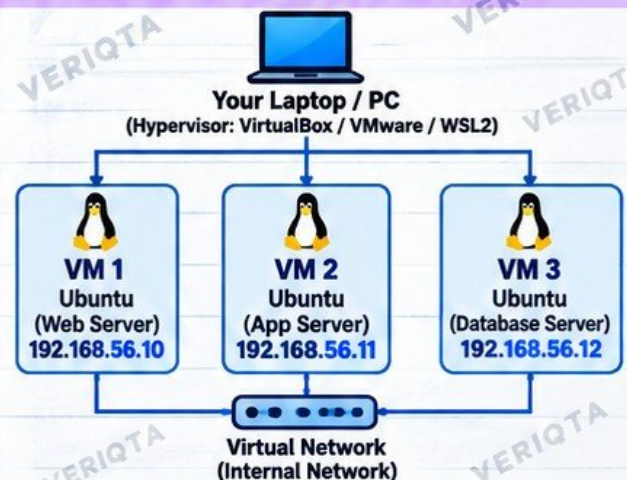


Hypervisor
Creates and manages virtual machines



Linux VMs
Your lab servers (eg. Ubuntu, CentOS, Rocky Linux, etc.)

4. FINAL LAB ARCHITECTURE



5. WHAT THE LEARNER WILL BUILD

- ✓ Set up a hypervisor (VirtualBox / VMware / WSL2)
- ✓ Create multiple Linux virtual machines
- ✓ Configure networking between VMs
- ✓ Perform basic system administration tasks
- ✓ Install and manage real services (e.g. Nginx, MySQL)
- ✓ Practice troubleshooting and monitoring
- ✓ Use the lab for future DevOps and cloud learning



JUNIOR ENGINEER TIP

Your home lab does not need to be powerful or expensive. Start small, be consistent, and add more servers and tools as you grow. What matters is hands-on practice.



REMEMBER THIS

A home lab is not just for learning Linux. It is the foundation for DevOps, cloud, networking, security, automation and more.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Practice
Build
Improve
Repeat

Choose Your Lab Platform

There are several ways to run Linux in your home lab. Choose the one that fits your goals, computer resources and learning style.

Page 2 of 20

1. POPULAR LAB PLATFORMS

VirtualBox



- ✓ Free and open source
- ✓ Easy to use
- ✓ Works on Windows, macOS and Linux
- ✓ Great for beginners

VMware Workstation Player



- ✓ Free for personal use
- ✓ Stable and reliable
- ✓ Good performance
- ✓ Works on Windows and Linux
- ✓ Suitable for multi-VM labs

WSL2 (Windows Subsystem for Linux)



- ✓ Runs Linux inside Windows
- ✓ No separate VM needed
- ✓ Lightweight and fast
- ✓ Great for development and CLI practice
- ✓ Not ideal for full infrastructure labs

Spare Hardware (Old Laptop / PC)



- ✓ Runs real Linux (no VM)
- ✓ Best performance
- ✓ Useful for advanced practice
- ✓ Can be used as a persistent home server
- ✓ Requires extra hardware

2. VM VERSUS PHYSICAL SERVER

Virtual Machine (VM)

- ✓ Runs on your laptop/PC using a hypervisor
- ✓ Easy to create, delete and snapshot
- ✓ Flexible and cost-effective
- ✓ Ideal for learning and testing
- ✓ Uses your existing hardware

Best for: Beginners and most home lab use cases.

Physical Server

- ✓ Runs directly on hardware
- ✓ Better performance
- ✓ More realistic experience
- ✓ Useful for advanced labs and 24/7 services
- ✓ Requires extra hardware and power

Best for: Advanced practice and always-on services.

3. RECOMMENDED BEGINNER SETUP

- ✓ Use VirtualBox or VMware on your laptop/PC.
- ✓ Create 1-3 Linux VMs (e.g. Ubuntu Server).
- ✓ Allocate enough CPU, RAM and disk (see below).
- ✓ Use a private network for VM-to-VM communication.
- ✓ Take snapshots before making big changes.
- ✓ Start small and expand as you grow.



JUNIOR ENGINEER TIP

You don't need an expensive setup to learn. A laptop with reasonable specs and a free tool like VirtualBox is enough to build real skills.

4. RESOURCE REQUIREMENTS (PER LINUX VM)

Resource	Minimum (OK)	Recommended	Notes
CPU (vCPU)	1	2	More CPU gives better performance
RAM	1 GB	2 - 4 GB	4 GB is more comfortable
Disk Space	20 GB	40 - 60 GB	Use a larger disk if you plan to install multiple tools
Network	NAT or Bridged	NAT + Host-Only	Host-Only allows VM-to-VM communication



REMEMBER THIS

The best platform is the one you will actually use. Start, learn, and focus on practice, not perfection.



YOUR TURN

Choose a platform, install it on your computer, and create your first Linux VM in the next page.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Same
Knowledge
Bigger
You

Build Your First Linux Server

In this lab, you will create your first Ubuntu Server virtual machine, configure the hardware settings, install the operating system and log in for the first time.

Page 3 of 20

1. CREATE AN UBUNTU SERVER VM

- 1 Open your virtualization tool (e.g. VirtualBox).
- 2 Click New to create a virtual machine.
- 3 Give it a name (e.g. Ubuntu-Server).
- 4 Select Type: Linux and Version: Ubuntu (64-bit).
- 5 Click Next and configure CPU, memory, disk and network settings (see next section).
- 6 Start the VM and attach the Ubuntu Server ISO file.
- 7 Proceed with the installation.

2. VM HARDWARE CONFIGURATION

Setting	Recommended (Beginner)	Notes
CPU (vCPU)	2	1 works, but 2 is smoother
Memory (RAM)	2 – 4 GB	2 GB minimum, 4 GB better
Disk Size	20 – 40 GB	Use 20 GB for basic lab
Network	NAT (default)	Good for internet access
(Optional)	Bridged Adapter	Use if you want your VM on your local network

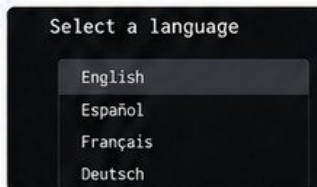


These settings are enough for a functional Linux server and most DevOps practice tasks.

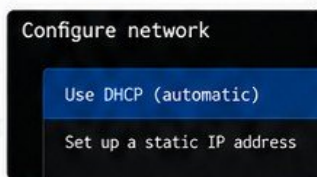
3. INSTALLATION WALKTHROUGH



- 1 Boot from the ISO file and select Install Ubuntu Server.



- 2 Choose your language (e.g. English).



- 3 Configure network (settings depend on your lab plan).



- 4 Create your user, set a hostname and choose a strong password.

4. FIRST BOOT AND LOGIN

```
Ubuntu 24.04 LTS ubuntu-server tty1
ubuntu-server login: veriqa
Password: *****
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-31-generic x86_64)

veriqa@ubuntu-server:~$
```



Log in with the username and password you created.



You are now on your Ubuntu Server.



Check your system information:

```
uname -a      # shows kernel version
hostnamectl   # shows hostname and OS info
ip addr       # shows your IP address
```

5. WHAT YOU HAVE ACHIEVED



A running Ubuntu Server virtual machine



Correct CPU, memory, disk and network configuration



A successful installation



A working login to your Linux server



You are ready to start exploring and installing tools



REMEMBER THIS

This is your first Linux server. It does not have to be perfect. The goal is to get it running and start practising.



YOUR TURN

Create your own Ubuntu Server VM now. Take a screenshot of your login terminal.



@veriqa
Instagram



@veriqa
GitHub



@veriqa
LinkedIn



@veriqa
Telegram



@veriqa
Medium



@veriqa
X

Consistent
Practice
Real
Progress

Understand Your Linux System

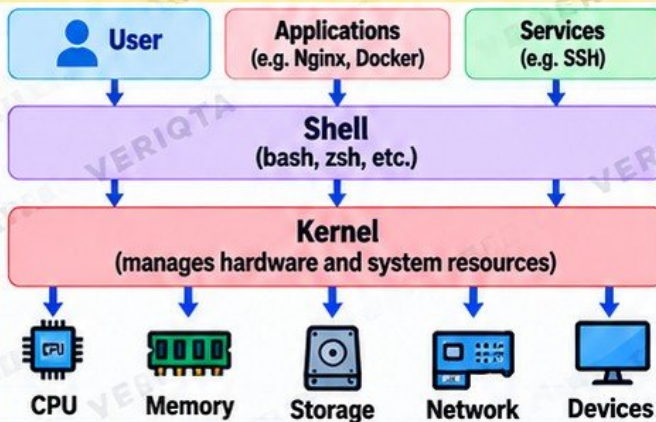
Linux is made up of several key components that work together. In this page, you will learn what they are, how to check them, and understand what is actually running on your system.

Page 4 of 20

1. KEY COMPONENTS OF LINUX

-  **Kernel** The core of Linux. It manages hardware resources (CPU, memory, storage, devices) and provides services to programs.
-  **Shell** A command-line interface that lets you interact with Linux. Examples: bash, zsh.
-  **Filesystem** How files and directories are organized. Everything in Linux is a file.
-  **Processes** Running programs with a unique Process ID (PID).
-  **Services** Background processes that provide functionality (e.g. SSH, Nginx). Managed by systemd on most modern Linux systems.

3. HOW EVERYTHING FITS TOGETHER



4. SEE WHAT IS ACTUALLY RUNNING

Use these commands to explore your running system.

\$ ps aux	Shows all running processes.
\$ top	Real-time view of system processes.
\$ systemctl list-units --type=service	Shows running services.
\$ df -h	Shows disk usage and filesystems.
\$ free -h	Shows memory usage.
\$ uptime	Shows how long the system has been running.

2. CHECK YOUR SYSTEM INFORMATION

Use these commands to understand your Linux system.

\$ uname -a Shows kernel information.

```
veriqta@linux:~$ uname -a
Linux linux-server 6.8.0-31-generic #31-Ubuntu SMP
x86_64 GNU/Linux
```

\$ hostnamectl Shows hostname and OS details.

```
veriqta@linux:~$ hostnamectl
Static hostname: linux-server
Operating System: Ubuntu 24.04 LTS
Kernel: Linux 6.8.0-31-generic
Architecture: x86-64
```

\$ lsb_release -a Shows distribution version.

```
veriqta@linux:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 24.04 LTS
Release: 24.04
Codename: noble
```

5. INTERPRET WHAT YOU SEE

- ✓ Kernel version tells you the OS and hardware compatibility.
- ✓ Hostname shows the server name (useful in multi-server labs).
- ✓ OS version tells you the Linux distribution and release.
- ✓ Running processes show what applications are active.
- ✓ Services show what system features are enabled.
- ✓ Disk and memory usage help you understand resource usage.
- ✓ Uptime shows system stability.



JUNIOR ENGINEER TIP

Get familiar with these commands. They are essential for troubleshooting, performance checks and daily administration. The more you explore your system, the more confident you become as a Linux engineer.



REMEMBER THIS

Linux is not just a command line. It is a complete operating system with many components working together. Understanding how they fit helps you troubleshoot, optimize and manage your servers better.

Master the Linux Filesystem

Linux uses a single directory tree that starts at the root (/). Understanding the filesystem structure helps you find files, install applications, and troubleshoot problems.

Page 5 of 20

1. KEY LINUX DIRECTORIES

Directory	Purpose	Contains / Used For
/	Root	Top of the filesystem. Everything starts here.
/etc	Configuration	System configuration files (e.g. nginx.conf, sshd_config).
/home	User files	Home directories for users (e.g. /home/veriqta).
/var	Variable data	Logs, databases, application data (e.g. /var/log).
/tmp	Temporary files	Temporary files (usually deleted on reboot).
/usr	User programs	Installed applications and libraries (e.g. /usr/bin, /usr/local).
/opt	Optional software	Third-party applications (e.g. /opt/docker).

3. USEFUL COMMANDS

Command	What It Does	Example	Sample Output
pwd	Show current directory	<code>pwd</code>	/home/veriqta
ls	List files and directories	<code>ls -l</code>	drwxr-xr-x 2 veriqta 4096 ...
cd	Change directory	<code>cd /etc</code>	(no output)
find	Search for files and directories	<code>find /var -name "*.log"</code>	/var/log/syslog /var/log/nginx/...

5. PRACTICAL EXAMPLES

Go to the /etc directory:	<code>\$ cd /etc</code>
List files in long format:	<code>\$ ls -l</code>
Find all .conf files in /etc:	<code>\$ find /etc -name "*.conf"</code>
Go to your home directory:	<code>\$ cd ~</code>
Show current directory:	<code>\$ pwd</code>

2. ABSOLUTE vs RELATIVE PATHS

Absolute Path

- Starts from the root (/)
- Shows the full path to a file or directory
- Always points to the same location

Example:

`/etc/nginx/nginx.conf`

Relative Path

- Starts from your current directory
- Does not include the full path
- Shorter and easy to use
- Depends on where you are in the filesystem

Example:

`nginx/nginx.conf`


TIP

Use absolute paths in scripts and automation.
Use relative paths for quick navigation.

4. WHERE THINGS LIVE


Applications

/usr/bin, /usr/sbin
/opt (third-party apps)
/var/www (web apps e.g. Nginx)


Configuration Files

/etc
(e.g. /etc/nginx/, /etc/ssh/)


Logs

/var/log
(e.g. /var/log/syslog,
/var/log/nginx/)


User Data

/home/username


Optional Software

/opt

6. REMEMBER THIS



- The filesystem starts at /.
- Configuration files are usually in /etc.
- Logs are usually in /var/log.
- Your personal files are in /home.
- Use absolute paths for scripts.
- Learn to navigate with `pwd`, `ls`, `cd` and `find`.



JUNIOR ENGINEER TIP

Spend time exploring the filesystem. Run `ls` and `cd` in different directories to understand what is installed on your system.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Same
Knowledge
Bigger
You

Files and Directories in Practice

In this page, you will learn how to create, manage, view, move and delete files and directories using common Linux commands. You will also build a realistic lab directory structure.

Page 6 of 20

1. CREATE FILES AND DIRECTORIES

Create an empty file



```
$ touch file.txt
```

Creates a new empty file.

Create a directory



```
$ mkdir projects
```

Creates a new directory.

Create multiple directories



```
$ mkdir -p devops/{scripts,configs,logs}
```

Creates parent directories.

2. COPY, MOVE AND REMOVE

Copy a file



```
$ cp file.txt backup.txt
```

Copies a file.

Copy a directory



```
$ cp -r projects/ projects_backup/
```

Copies a directory (recursive).

Move or rename a file



```
$ mv file.txt notes.txt
```

Moves or renames a file.

Move a directory



```
$ mv projects/ /opt/
```

Moves a directory.

Remove a file



```
$ rm file.txt
```

Deletes a file (permanent).

Remove a directory



```
$ rm -r projects/
```

Deletes a directory and its contents.

3. VIEW FILE CONTENTS

Show full content



```
$ cat file.txt
```

Displays the entire file content.

View with scrolling



```
$ less file.txt
```

View file content page by page (press q to exit).

Show the first lines



```
$ head -n 10 file.txt
```

Shows the first 10 lines (default is 10).

Show the last lines



```
$ tail -n 10 file.txt
```

Shows the last 10 lines (use -f to follow in real time).

4. SAFE DELETION HABITS

- ✓ Always double-check the path before using rm.
- ✓ Use ls to confirm what you are deleting.
- ✓ Avoid using rm -rf unless you are 100% sure.
- ✓ Consider moving files to a backup directory instead of deleting immediately.
- ✓ Use the trash (e.g. trash-cli) for an extra layer of safety (if available).
- ✓ Be especially careful with /, /etc, /var and other system directories.
- ✓ Use tab completion to avoid typing mistakes.
- ✓ When in doubt, stop and verify.



"A careful engineer works today and still has access tomorrow."

5. BUILD A REALISTIC LAB DIRECTORY STRUCTURE

```
$ mkdir -p ~/linux-lab/{projects,configs,logs,scripts,backups}
$ tree ~/linux-lab
/home/veriqta/linux-lab
├── backups
├── configs
├── logs
├── projects
└── scripts
5 directories, 0 files
```

This structure can be used to:

- ✓ Store your practice projects
- ✓ Keep configuration files
- ✓ Save logs
- ✓ Write your scripts
- ✓ Keep backups
- ✓ Use it for future DevOps labs



JUNIOR ENGINEER TIP

A well-organized directory structure saves time, reduces mistakes and makes your work look professional. Use clear and consistent names.



PRACTICE TASK

Create the linux-lab directory structure shown on the left. Then, create a file in each directory and view its content using cat, head and less.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Users, Groups, and sudo

Linux uses users and groups to control access and permissions. In this page, you will learn the difference between root and normal users, explore `/etc/passwd` and `/etc/group`, and practice creating and managing users with `sudo`.

1. ROOT vs NORMAL USERS

Root User



- Username: root
- Highest level of access
- Can do anything on the system
- Used for system administration
- Should not be used for daily work
- Mistakes can break the system

Normal User



- Regular user account
- Limited permissions
- Used for daily activities
- Safer for everyday work
- Can use `sudo` to run administrative commands
- Recommended for DevOps practice

3. USER MANAGEMENT COMMANDS

Command	Description	Example
<code>useradd</code>	Create a new user	<code>sudo useradd devops</code>
<code>usermod</code>	Modify an existing user	<code>sudo usermod -aG sudo devops</code>
<code>groups</code>	Show groups for a user	<code>groups devops</code>
<code>id</code>	Show user and group IDs	<code>id devops</code>
<code>passwd</code>	Set or change password	<code>sudo passwd devops</code>
<code>deluser</code>	Delete a user	<code>sudo deluser devops</code>

5. CREATE A DEDICATED DEVOPS USER

Let's create a dedicated DevOps user and give `sudo` access.

- 1 Create the user `$ sudo useradd -m -s /bin/bash devops`
- 2 Set a password `$ sudo passwd devops`
- 3 Add to `sudo` group `$ sudo usermod -aG sudo devops`
- 4 Verify the user `$ id devops`
- 5 Test `sudo` access `$ sudo whoami`



REMEMBER THIS

Do not use the root account for daily work.
Create a dedicated user, give it `sudo` access, and work safely.

2. /ETC/PASSWD AND /ETC/GROUP



`/etc/passwd`

Contains information about all users on the system.

Format: username:x:UID:GID:comment:home_directory:shell

```
$ cat /etc/passwd | head -n 3
root:x:0:0:root:/root:/bin/bash
veriqta:x:1000:1000:Veriqta User:/home/veriqta:/bin/bash
devops:x:1001:1001:DevOps User:/home/devops:/bin/bash
```



`/etc/group`

Contains information about groups on the system.

Format: groupname:x:GID:member1,member2,...

```
$ cat /etc/group | head -n 3
root:x:0:
sudo:x:27:veriqta
devops:x:1001:devops
```

4. UNDERSTANDING sudo



sudo (superuser do) allows a normal user to run administrative commands with elevated privileges.

- ✓ Add a user to the `sudo` group to give admin access.
- ✓ Always use `sudo` instead of logging in as root.
- ✓ Keeps your system safer and provides an audit trail.
- ✓ You may be asked for your password when using `sudo`.

Example: Run a command with `sudo`

```
$ sudo apt update
```

6. VERIFY AND SWITCH USER

Switch to devops user	<code>\$ su - devops</code>
Check current user	<code>\$ whoami</code>
Check groups	<code>\$ groups</code>
Run an admin command	<code>\$ sudo apt update</code>



JUNIOR ENGINEER TIP

Use a normal user for daily work and `sudo` for administrative tasks. This is the standard practice in real-world Linux environments.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Same
Knowledge
Bigger
You

Linux Permissions Without chmod 777

In this page, you will learn how Linux permissions work, understand owner, group and others, use chmod, chown and chgrp, work with numeric permissions, see why 777 is usually the wrong fix, and configure correct permissions for SSH keys.

Page 8 of 20

1. UNDERSTAND PERMISSIONS

Every file and directory has three types of permissions:

Permission	Symbol	Meaning
 Read	r	View the contents (files) or list contents (directories)
 Write	w	Modify the contents (files) or add/remove files (directories)
 Execute	x	Run the file (program) or enter the directory

2. OWNER, GROUP, OTHERS

Each file and directory has an owner, a group and others.

```
$ ls -l file.txt
-rw-r--r-- 1 veriqa devops 1024 Sep 14 10:00 file.txt
```

Owner
(veriqa)

First 3 permissions
(rw-)

Group
(devops)

Next 3 permissions
(r--)

Others
(everyone else)

Last 3 permissions
(r--)

3. chmod, chown AND chgrp

Common commands:

chmod	Change permissions	chmod 755 script.sh
chown	Change owner	chown veriqa file.txt
chgrp	Change group	chgrp devops file.txt
chown (recursive)	Change owner for a directory and its contents	chown -R veriqa:devops project/

4. NUMERIC PERMISSIONS

Each permission has a number: **r = 4, w = 2, x = 1**

Permission	Value	Common examples:
r (read)	4	chmod 755 file.sh rwx r-x r-x
w (write)	2	chmod 644 file.txt rw- r-- r--
x (execute)	1	chmod 700 script.sh rwx --- ---
		chmod 600 file.txt rw- --- ---

5. WHY chmod 777 IS USUALLY THE WRONG FIX



- Gives full access (read, write, execute) to everyone.
- Creates serious security risks.
- Allows anyone on the system to modify or run the file.
- Can be exploited by malicious users or scripts.
- Not required in most real-world scenarios.
- Use the minimum permissions needed (principle of least privilege).

"If it works with 777, find the real solution, don't keep 777."

6. SSH-KEY PERMISSION EXAMPLE

SSH keys must have strict permissions or SSH will refuse to use them.

- 1 Check current permission

```
$ ls -l ~/.ssh/id_rsa
-rw-r--r-- 1 veriqa veriqa 1675 Sep 14 10:00 id_rsa
```
- 2 Set correct permission

```
$ chmod 600 ~/.ssh/id_rsa
```
- 3 Set directory permission

```
$ chmod 700 ~/.ssh
```
- 4 Verify

```
$ ls -l ~/.ssh/id_rsa
-rw----- 1 veriqa veriqa 1675 Sep 14 10:00 id_rsa
```

7. PRACTICAL EXERCISE

- ✓ Create a file and set different permissions (644, 755, 600).
- ✓ Create a directory and change its owner and group.
- ✓ Build a small project folder and set appropriate permissions.
- ✓ Configure correct permissions for an SSH key pair.
- ✓ Verify and explain what each permission means.

8. KEY TAKEAWAYS

- ✓ Understand read, write and execute.
- ✓ Know the difference between owner, group and others.
- ✓ Use chmod, chown and chgrp confidently.
- ✓ Use numeric permissions.
- ✓ Avoid chmod 777.
- ✓ Set correct permissions for SSH keys.
- ✓ Always use the minimum permissions required.



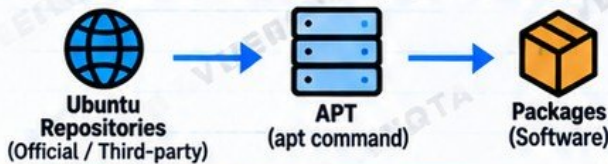
Packages and Software Management

Linux distributions use package managers to install, update and manage software. In this page, you will learn about repositories, common apt commands, install useful tools, and verify your installations.

Page 9 of 20

1. REPOSITORIES AND PACKAGES

- A **package** is a software bundle that contains the application, its dependencies and metadata.
- A **repository** is an online storage location that holds packages.
- Your system uses configured repositories to download and install packages.
- Ubuntu uses the APT (Advanced Package Tool) package manager.



Common Ubuntu repositories:

Repository	Purpose	Example
Main	Officially supported free software	Ubuntu main repo
Universe	Community maintained software	e.g. tools, utilities
Multiverse	Software with restricted licenses	e.g. drivers, codecs
PPA	Personal Package Archive (third party)	e.g. nginx, git (newer)

3. INSTALL USEFUL TOOLS

Let's install some commonly used tools for DevOps and system administration.

```
$ sudo apt update # Update package list
```

```
$ sudo apt install -y git curl nginx \
net-tools iproute2 vim htop tree
```

Install multiple useful tools.

Tool	Purpose	Verify Command
git	Version control system	git --version
curl	Transfer data from URLs	curl --version
nginx	Web server	nginx -v
net-tools	Network troubleshooting tools	ifconfig
iproute2	Modern networking tools	ip -version
vim	Text editor	vim --version
htop	System monitoring (interactive)	htop --version
tree	Display directory structure	tree --version

6. KEY TAKEAWAYS

- ✓ Repositories provide access to software packages.
- ✓ Use apt update before installing or upgrading.
- ✓ Use apt install to install new software.
- ✓ Verify installations using version checks and package information.
- ✓ Use troubleshooting steps when you encounter errors.

2. ESSENTIAL APT COMMANDS

Update package list

```
$ sudo apt update
```

Downloads the latest package information.

Upgrade installed packages

```
$ sudo apt upgrade -y
```

Installs the latest versions of installed packages.

Install a package

```
$ sudo apt install package_name
```

Installs a new package.

Remove a package

```
$ sudo apt remove package_name
```

Removes a package (keeps config files).

Remove a package and config files

```
$ sudo apt purge package_name
```

Removes package and its configuration files.

Search for a package

```
$ apt search package_name
```

Searches for a package in the repositories.

4. VERIFY INSTALLED SOFTWARE

You can verify that a package is installed using different methods.

1. Check version

```
$ git --version
```

Shows the installed version.

2. Check package details

```
$ apt show nginx
```

Shows detailed information.

3. List installed packages

```
$ apt list --installed | grep nginx
```

Confirms if the package is installed.

4. Check binary location

```
$ which nginx
```

Shows the path to the executable.

5. COMMON TROUBLESHOOTING

Issue	Solution
Package not found	Run <code>sudo apt update</code> and check the package name.
Permission denied	Use <code>sudo</code> before apt commands.
Network/Repository error	Check your internet connection and repository settings in <code>/etc/apt/sources.list</code> .
Broken packages	Run <code>sudo apt --fix-broken install</code> .
Held packages	Run <code>sudo apt upgrade</code> or check held packages with <code>apt-mark showhold</code> .



JUNIOR ENGINEER TIP

Keep your system updated regularly. Updated packages include security fixes, bug fixes and new features.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Learn
Build
Improve
Repeat

Processes and Resource Management

In this page, you will learn how Linux processes work, understand process lifecycle, use common commands to view and manage processes, investigate CPU and memory usage, and troubleshoot misbehaving processes.

Page 10 of 20

1. PROCESS LIFECYCLE

A process goes through different states during its lifetime.



- A process is created using `fork()` and starts execution.
- It moves between ready, running and waiting states.
- It terminates when it completes or is killed.

2. PID AND PARENT PROCESSES

- Every process has a unique Process ID (PID).
- Every process (except the first) has a parent process (PPID).
- You can view the process tree to see the relationship between processes.

`$ ps -ef | head -5`

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	08:00	?	00:00:02	/sbin/init
root	234	1	0	08:00	?	00:00:00	/usr/sbin/sshd
veriqta	1024	234	0	08:01	pts/0	00:00:00	-bash
veriqta	2048	1024	0	08:05	pts/0	00:00:00	ps -ef



TIP PID 1 is usually the `init` or `systemd` process. Use the PPID to understand which process started another process.

3. COMMON PROCESS COMMANDS

Command	Description	Example
<code>ps</code>	View running processes	<code>ps aux</code>
<code>top</code>	Interactive process monitor	<code>top</code>
<code>pgrep</code>	Find process by name	<code>pgrep nginx</code>
<code>kill</code>	Send signal to a process	<code>kill 1234</code>
<code>kill -9</code>	Force kill a process	<code>kill -9 1234</code>
<code>pstree</code>	Show process tree	<code>pstree -p</code>
<code>nice</code>	Start a process with priority	<code>nice -n 10 ./script.sh</code>
<code>renice</code>	Change process priority	<code>renice -n 5 -p 1234</code>

4. CPU AND MEMORY INVESTIGATION

Find out which processes are using CPU and memory.



CPU USAGE

```
# Real-time view
$ top

# Sorted by CPU
$ ps aux --sort=-%cpu | head

# Find high CPU process
$ pgrep -l -f process_name
```



MEMORY USAGE

```
# Real-time view
$ top

# Sorted by memory
$ ps aux --sort=-%mem | head

# Detailed memory info
$ free -h

# Show memory usage by process
$ pmap -x <PID>
```

5. TROUBLESHOOT A MISBEHAVING PROCESS

Follow these steps when a process is using too much CPU, memory or is not behaving as expected.

- 1 Identify the process `$ ps aux | grep process_name`
- 2 Investigate resource usage `$ top (or htop)`
- 3 Check process details `$ ps -fp <PID>`
- 4 Check open files `$ lsof -p <PID>`
- 5 Check what it is doing `$ strace -p <PID>`
- 6 Terminate if necessary `$ kill <PID> (or kill -9 <PID>)`

6. PRACTICAL EXAMPLE

A simple example of finding and stopping a high CPU process.

- 1 Find the process using high CPU `$ ps aux --sort=-%cpu | head`
- 2 Note the PID (e.g. 5678)
- 3 See more details `$ ps -fp 5678`
- 4 Check what files it is using `$ lsof -p 5678`
- 5 Stop the process `$ kill 5678`
- 6 If it does not stop `$ kill -9 5678`

7. KEY TAKEAWAYS

- ✓ Understand the process lifecycle.
- ✓ Know how to find and analyze processes.
- ✓ Use the right tools to investigate CPU and memory usage.
- ✓ Be careful when killing processes.
- ✓ Troubleshoot systematically before taking action.



JUNIOR ENGINEER TIP

Do not kill a process unless you understand its purpose. Investigate first. A misbehaving process can be a symptom of a bigger issue.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Learn
Build
Improve
Repeat

Services with systemd

In this page, you will learn how services work in Linux using systemd. You will understand the difference between a service and a process, learn common systemctl commands, use journalctl to view logs, start Nginx automatically at boot, and troubleshoot failed services.

1. SERVICE VS PROCESS

Process

- A process is a running program with a unique PID.
- It can be started manually.
- It runs in the foreground or background.
- It may stop if the terminal closes.
- Example: nginx (process)



Process = Running program (PID)

Service (systemd)

- A service is a managed process controlled by systemd.
- systemd can start, stop, restart and monitor the process.
- It can be configured to start automatically at boot.
- It keeps the process running.
- Example: nginx.service (service)



Service = Managed process (by systemd)

2. COMMON SYSTEMCTL COMMANDS

Manage services using systemctl.

Command	Description	Example (Nginx)
<code>systemctl status</code>	Check service status	<code>systemctl status nginx</code>
<code>systemctl start</code>	Start a service	<code>systemctl start nginx</code>
<code>systemctl stop</code>	Stop a service	<code>systemctl stop nginx</code>
<code>systemctl restart</code>	Restart a service	<code>systemctl restart nginx</code>
<code>systemctl enable</code>	Enable at boot	<code>systemctl enable nginx</code>
<code>systemctl disable</code>	Disable at boot	<code>systemctl disable nginx</code>
<code>systemctl is-active</code>	Check if active	<code>systemctl is-active nginx</code>
<code>systemctl is-enabled</code>	Check if enabled	<code>systemctl is-enabled nginx</code>

3. USING JOURNALCTL

Use journalctl to view logs from systemd services.

```
# View all logs for a service
$ journalctl -u nginx

# Follow logs in real time
$ journalctl -u nginx -f

# View recent logs (last 50 lines)
$ journalctl -u nginx -n 50

# View logs since a specific time
$ journalctl -u nginx --since "1 hour ago"

# View logs with errors only
$ journalctl -u nginx -p err
```



journalctl shows logs from systemd and is essential for troubleshooting failed services.

4. START NGINX AUTOMATICALLY AT BOOT

Enable and start Nginx so it runs after a system reboot.

- 1 Install Nginx (if not installed)

```
$ sudo apt install nginx
```

- 2 Start Nginx

```
$ sudo systemctl start nginx
```

- 3 Enable Nginx at boot

```
$ sudo systemctl enable nginx
```

- 4 Verify

```
$ systemctl is-enabled nginx
```



Enabling a service ensures it starts automatically after a reboot.

5. DIAGNOSE A FAILED SERVICE

Steps to troubleshoot a service that is not working.

- 1 Check service status `$ systemctl status nginx`
- 2 View logs for errors `$ journalctl -u nginx -n 50`
- 3 Check configuration (if applicable) `$ sudo nginx -t`
- 4 Check if the process is running `$ ps aux | grep nginx`
- 5 Restart the service `$ sudo systemctl restart nginx`
- 6 Check system resources `$ free -h # memory`
`$ df -h # disk space`

6. COMMON SERVICE FILE LOCATION

Service unit files are usually stored in:

```
/etc/systemd/system/ # custom services
/lib/systemd/system/ # packaged services (e.g. nginx)
```

7. KEY TAKEAWAYS

- ✓ Understand the difference between a service and a process.
- ✓ Use systemctl to manage services.
- ✓ Use journalctl to view and troubleshoot logs.
- ✓ Enable important services (e.g. Nginx) at boot.
- ✓ Follow a structured approach to diagnose failed services.



JUNIOR ENGINEER TIP

When a service fails, always check the logs. The error message usually tells you exactly what went wrong.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Learn
Build
Improve
Repeat

Linux Networking Lab

In this page, you will learn Linux networking fundamentals, how to inspect and manage network configuration, test connectivity, work with DNS, and understand how traffic reaches your VM.

Page 12 of 20

1. NETWORKING FUNDAMENTALS

Key concepts you need to understand:

IP Address	Identifies your machine on the network. (IPv4 and IPv6)
Interface	Network devices like eth0, ens33, lo (loopback).
Ports	Used by applications (e.g. 22 for SSH, 80 for HTTP, 443 for HTTPS).
Routes	Decide where network traffic goes.
DNS	Converts domain names (e.g. google.com) to IP addresses.

3. ROUTES AND GATEWAY

View your routing table.

```
$ ip route
default via 192.168.1.1 dev ens33
192.168.1.0/24 dev ens33 proto kernel scope link src 192.168.1.100
```



Route Type	Meaning
default	Traffic to other networks (uses gateway)
192.168.1.0/24	Local network
via 192.168.1.1	Your gateway (usually your router)
dev ens33	Network interface used

6. DNS LOOKUP EXAMPLE

Use dig to query DNS records.

```
$ dig google.com
;; ANSWER SECTION:
google.com. 300 IN A 142.250.72.14
google.com. 300 IN A 142.250.72.46
```

Resolves domain names to IP addresses and shows DNS details.

8. PRACTICAL LAB TASKS

- Find your VM's IP address.
- Check your default gateway and routes.
- Test connectivity to another server (e.g. ping 8.8.8.8).
- Use curl to access a website (e.g. curl https://google.com).
- Use dig to query a domain (e.g. dig veriqta.com).
- Check which ports are open on your VM.

2. CHECK IP ADDRESSES AND INTERFACES

View your network interfaces and IP addresses.

```
$ ip addr
1: lo: <LOOPBACK> mtu 65536 state UNKNOWN
   inet 127.0.0.1/8 scope host lo
2: ens33: <BROADCAST,MULTICAST,UP> mtu 1500 state UP
   inet 192.168.1.100/24 brd 192.168.1.255 scope global ens33
```

Shows all network interfaces and IP addresses.

4. CHECK OPEN PORTS

View listening ports and active connections.

```
$ ss -tulnp
Netid State Local Address:Port Process
tcp LISTEN 0.0.0.0:22 sshd
tcp LISTEN 0.0.0.0:80 nginx
tcp LISTEN 127.0.0.1:3306 mysql
```

Shows open ports, listening services and the processes using them.

5. TEST CONNECTIVITY

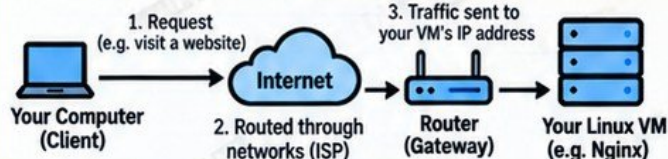
Use these commands to test network connectivity.

Command	Purpose
\$ ping <host>	Check if a host is reachable.
\$ curl <url>	Test HTTP/HTTPS access.
\$ dig <domain>	Query DNS records.
\$ ss -tulnp	View listening ports.



Test
Connect
Troubleshoot

7. HOW TRAFFIC REACHES YOUR VM



9. KEY TAKEAWAYS

- Understand IP addresses, interfaces, ports, routes and DNS.
- Use ip addr, ip route, ss, ping, curl and dig.
- Know how traffic flows from your computer to your VM.
- Be able to troubleshoot basic networking issues.
- Networking knowledge is essential for DevOps, Cloud and SRE roles.



JUNIOR ENGINEER TIP

When something is not working, always check:
1. Is the VM reachable? 2. Is the port open?
3. Is DNS resolving? 4. Is the service running?



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Learn
Build
Improve
Repeat

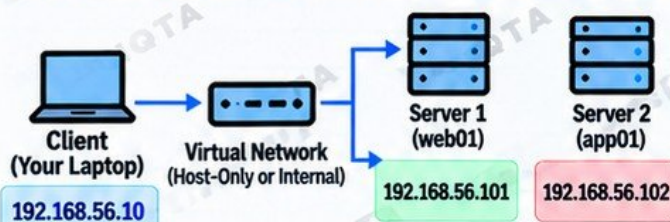
Build a Multi-Server Home Lab

In this page, you will set up a simple multi-server home lab with two Linux servers. You will configure hostnames and private IPs, test VM-to-VM communication, understand client → network → server flow, and verify connectivity between all machines.

Page 13 of 20

1. LAB OVERVIEW

We will create a simple network with two Linux servers and a client.



- ✓ All machines are on the same private network.
- ✓ They can communicate with each other.
- ✓ You will test connectivity using ping, curl and SSH.

2. CREATE SERVER 1 AND SERVER 2

Create two Linux VMs (Ubuntu Server recommended).

Server 1 Configuration

Hostname:	web01
IP Address:	192.168.56.101
OS:	Ubuntu Server
Role:	Web Server (Nginx)

Server 2 Configuration

Hostname:	app01
IP Address:	192.168.56.102
OS:	Ubuntu Server
Role:	Application Server (e.g. simple app)

Set the hostname (on each server)

```
$ sudo hostnamectl set-hostname web01 # on Server 1
$ sudo hostnamectl set-hostname app01 # on Server 2
```

Verify:

```
$ hostnamectl
$ hostname
```

3. CONFIGURE PRIVATE IP ADDRESSES

Set static IP addresses (example using Netplan on Ubuntu).

```
# Example for Server 1 (web01)
$ sudo nano /etc/netplan/01-network-manager-all.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: no
      addresses: [192.168.56.101/24]
      gateway4: 192.168.56.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Apply changes:
\$ sudo netplan apply

Do the same on
Server 2 with IP
192.168.56.102

4. TEST VM-TO-VM COMMUNICATION

From Server 1, ping Server 2 and vice versa.

```
# From web01 to app01
$ ping 192.168.56.102
# From app01 to web01
$ ping 192.168.56.101
```



You should see
successful ping
responses.

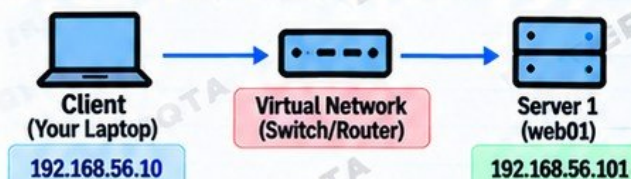
Test SSH (optional)

```
# From web01 to app01
$ ssh app01
```

Make sure SSH is installed
and the target server allows
connections.

5. CLIENT → NETWORK → SERVER FLOW

Understand how a request from your client reaches a server.



Test from your client machine:

```
$ ping 192.168.56.101 # Ping Server 1
$ ssh web01 # SSH to Server 1
```

This flow is the same
concept used in real
cloud and production
environments.



JUNIOR ENGINEER TIP

A home lab is a safe place to learn, break things and practice. The same networking concepts apply in real cloud environments (AWS, Azure, GCP).

6. ADD A SIMPLE SERVICE (OPTIONAL)

Install Nginx on Server 1 and test from your client or Server 2.

```
# On Server 1 (web01)
$ sudo apt update
$ sudo apt install -y nginx
$ sudo systemctl enable nginx --now
```

Test from Server 2
or your client:
\$ curl 192.168.56.101
You should see the
Nginx welcome page
(HTML output).

7. KEY TAKEAWAYS

- ✓ You can create multiple Linux servers on a private network.
- ✓ Each server has a unique hostname and private IP address.
- ✓ VMs can communicate with each other.
- ✓ You understand the client → network → server flow.
- ✓ You can test connectivity using ping, SSH and curl.
- ✓ This lab builds the foundation for real DevOps and cloud environments.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

SSH and Remote Administration

In this page, you will learn how SSH works, how to connect to a remote server, generate and use SSH keys, understand public and private keys, configure authorized_keys, secure key permissions, and stop relying on passwords for server access.

Page 14 of 20

1. HOW SSH WORKS

SSH (Secure Shell) provides an encrypted connection between your local machine and a remote server.



- Encrypts all data between client and server.
- Authenticates the user (password or SSH key).
- Allows secure remote administration.
- Runs on port 22 (by default).

SSH keeps your credentials, commands and data encrypted.

2. CONNECT TO A SERVER

Use the ssh command to connect to a remote server.

```
$ ssh user@server_ip
$ ssh ubuntu@192.168.56.101
```

Replace user with your username and server_ip with the server's IP address.

First time connection

```
$ ssh ubuntu@192.168.56.101
```

The authenticity of host '192.168.56.101' can't be established.
Are you sure you want to continue connecting (yes/no)? yes

You will be asked to confirm the server's fingerprint the first time.

3. GENERATE SSH KEYS

Generate a new SSH key pair on your local machine.

```
$ ssh-keygen -t ed25519 -C "your_email@example.com"
```

This creates a public key and a private key:

- Private key: ~/.ssh/id_ed25519 (keep this secret)
- Public key: ~/.ssh/id_ed25519.pub (share this with the server)



TIP

Press Enter to accept the default location and you can set a passphrase for extra security.

4. PUBLIC VS PRIVATE KEYS

Private Key



- Stored on your local machine.
- Must be kept secret.
- Used to prove your identity when connecting.
- If someone gets your private key, they can access your server.

Public Key



- Shared with the remote server.
- Added to the authorized_keys file.
- It does not give access on its own.
- Can be shared safely with multiple servers.

SSH uses a key pair. You authenticate with your private key, and the server verifies it using your public key.

5. ADD PUBLIC KEY TO SERVER

Copy your public key to the server.

```
$ ssh-copy-id ubuntu@192.168.56.101
```

This automatically adds your public key to ~/.ssh/authorized_keys on the server.

Or do it manually:

```
$ cat ~/.ssh/id_ed25519.pub
$ ssh ubuntu@192.168.56.101
$ mkdir -p ~/.ssh
$ nano ~/.ssh/authorized_keys # paste your public key
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/authorized_keys
```

The public key is stored in ~/.ssh/authorized_keys on the server.

6. SECURE KEY PERMISSIONS

Set correct permissions for SSH files and directories.

```
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/authorized_keys
$ chmod 600 ~/.ssh/id_ed25519
$ chmod 644 ~/.ssh/id_ed25519.pub
```

Incorrect permissions can cause SSH to refuse the key.



7. STOP RELYING ON PASSWORDS

- Use SSH keys instead of passwords.
- Disable password authentication on the server.
- This improves security and prevents brute-force attacks.

```
# Edit SSH config on the server
$ sudo nano /etc/ssh/sshd_config
```

Set this:
PasswordAuthentication no
ChallengeResponseAuthentication no
Then restart SSH:
\$ sudo systemctl restart ssh

8. KEY TAKEAWAYS

- ✓ Understand how SSH works.
- ✓ Use ssh user@server to connect.
- ✓ Generate and use SSH keys.
- ✓ Know the difference between public and private keys.
- ✓ Configure authorized_keys and set secure permissions.
- ✓ Stop relying on passwords for server access.



JUNIOR ENGINEER TIP

Use SSH keys for all your servers, including cloud VMs. It is more secure, faster and easier to manage than passwords.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Learn
Build
Improve
Repeat

Firewalls and Port Troubleshooting

In this page, you will learn how firewalls work, the difference between listening and reachable ports, use UFW (Uncomplicated Firewall), allow only required services, check open ports, and troubleshoot common connection issues like "Connection refused" and timeouts.

Page 15 of 20

1. LISTENING PORTS VS REACHABLE PORTS

Listening Port

- A service is running and listening on a port.
- It accepts connections on the local machine (127.0.0.1 or the server's IP).
- Does not mean it is reachable from other machines.

Reachable Port

- A port is listening and can be reached from another machine over the network.
- Firewall rules, security groups and network configuration must allow it.
- A port can be listening but not reachable if a firewall is blocking it.



Key Point: Listening ≠ Reachable

A service can be running, but still not reachable from outside.

2. UFW (UNCOMPLICATED FIREWALL) BASICS

UFW is a simple firewall for Ubuntu.

```
# Check status
$ sudo ufw status

# Enable UFW
$ sudo ufw enable

# Disable UFW
$ sudo ufw disable

# Reset rules
$ sudo ufw reset
```

Common UFW Commands

```
sudo ufw status
sudo ufw enable
sudo ufw disable
sudo ufw allow <port>
sudo ufw allow <service>
sudo ufw deny <port>
sudo ufw delete <rule>
sudo ufw reset
```



When UFW is enabled, only allowed traffic can reach your server. Always allow only the services you need.

3. ALLOW ONLY REQUIRED SERVICES

Allow specific services or ports.

Service	Command	Description
SSH (port 22)	<code>sudo ufw allow ssh</code>	Allow SSH access
HTTP (port 80)	<code>sudo ufw allow 80</code>	Allow web traffic
HTTPS (port 443)	<code>sudo ufw allow 443</code>	Allow secure web traffic
Nginx (by name)	<code>sudo ufw allow nginx</code>	Allow Nginx (80,443)
Custom port	<code>sudo ufw allow 8080</code>	Allow a specific port
Remove rule	<code>sudo ufw delete allow 8080</code>	Remove a rule
View numbered	<code>sudo ufw status numbered</code>	See rules with numbers

4. CHECK LISTENING PORTS

Use ss to view listening ports and the processes using them.

```
# Show all listening ports
$ ss -tulpn

# Show only TCP ports
$ ss -tulpen | grep tcp

# Show only a specific port (e.g. 80)
$ ss -tulpn | grep :80
```

-t = TCP
-u = UDP
-l = listening
-p = show process name (requires sudo)
-n = show numeric addresses and ports



If a service is not listening, it cannot be reachable. Check that the application is running and listening first.

5. TEST CONNECTIVITY

Use these tools to test if a port is reachable.

Command	What it does
<code>ping <ip></code>	Check if the host is reachable
<code>curl http://<ip></code>	Test HTTP service
<code>curl -I http://<ip></code>	Test HTTP without downloading content
<code>nc -zv <ip> 80</code>	Test if a port is open (netcat)
<code>telnet <ip> 80</code>	Test connection to a port (if installed)

6. TROUBLESHOOT COMMON ISSUES

Connection refused

- Service is not running.
- Port is not listening.
- Firewall is blocking the port.
- Wrong IP address or port.
- Application is bound to 127.0.0.1 (localhost) only.

```
$ curl http://192.168.56.101:80
curl: (7) Failed to connect:
Connection refused
```

Connection timeouts

- Host is unreachable (network issue).
- Firewall is blocking the traffic (silent drop).
- Wrong IP address.
- Cloud security group is blocking it.
- Routing or DNS issue.

```
$ curl http://192.168.56.102:80
curl: (28) Connection timed out
after 10001 milliseconds
```

7. KEY TAKEAWAYS

- ✓ Understand the difference between listening and reachable ports.
- ✓ Use UFW to allow only required services.
- ✓ Always check if a service is running and listening.
- ✓ Use ss, ping, curl and other tools to test connectivity.
- ✓ Troubleshoot systematically when you see connection refused or timeouts.



JUNIOR ENGINEER TIP

A secure server is not about opening more ports. It is about allowing only what you need and knowing how to troubleshoot when something does not work.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Learn
Build
Improve
Repeat

Logs and Linux Troubleshooting

In this page, you will learn where logs are stored, how to use journalctl and tail -f, and how to follow evidence instead of guessing. You will also troubleshoot a deliberately broken service using real logs.

Page 16 of 20

1. WHERE LINUX STORES LOGS

Most logs are stored in /var/log.

```
$ ls -l /var/log
```

total 56

```
drwxr-xr-x 2 root root 4096 Aug 26 10:00 apt/
-rw-r--r-- 1 root root 12345 Aug 26 10:00 auth.log
-rw-r--r-- 1 root root 23456 Aug 26 10:00 syslog
-rw-r--r-- 1 root root 3456 Aug 26 10:00 kern.log
-rw-r--r-- 1 root root 7890 Aug 26 10:00 nginx/
drwxr-xr-x 2 root root 4096 Aug 26 10:00 journal/
```

Common log files

- /var/log/syslog
System logs
- /var/log/auth.log
Authentication logs
- /var/log/kern.log
Kernel logs
- /var/log/apt/
Package management logs
- /var/log/nginx/
Nginx logs
- /var/log/journal/
Systemd journal logs

2. USING JOURNALCTL

journalctl is used to view logs from systemd services.

```
# View all logs
$ journalctl

# View logs for a specific service
$ journalctl -u nginx

# Follow logs in real time
$ journalctl -u nginx -f

# Show only recent logs (last 50 lines)
$ journalctl -u nginx -n 50

# Show logs since a specific time
$ journalctl -u nginx --since "1 hour ago"

# Show logs with errors only
$ journalctl -u nginx -p err
```

Useful options

- -u Specify service
- -f Follow logs
- -n Number of lines
- --since From time
- -p Log level
- -xe Detailed output (includes errors)



Use journalctl for systemd services (e.g. nginx, ssh, cron, etc.).

3. USING TAIL -F

Use tail -f to follow log files in real time.

```
# Follow a log file
$ tail -f /var/log/syslog

# Follow Nginx access log
$ tail -f /var/log/nginx/access.log

# Follow Nginx error log
$ tail -f /var/log/nginx/error.log

# Show last 50 lines
$ tail -n 50 /var/log/syslog
```

Press **Ctrl + C** to stop following logs.



tail -f is useful for application logs and any file that continuously receives new logs.

4. FOLLOW EVIDENCE, NOT GUESSING

When troubleshooting, always check the logs instead of guessing.

- ✓ Check service status.
- ✓ Look at recent logs.
- ✓ Identify error messages.
- ✓ Understand the root cause.
- ✓ Make the fix.
- ✓ Verify the service is working.

*"Logs tell the truth.
Don't guess.
Follow the evidence."*

- VERIQTA

5. TROUBLESHOOT A DELIBERATELY BROKEN SERVICE

In this lab, we will break Nginx on purpose and fix it using logs.

- 1 Stop Nginx.
- 2 Break the Nginx config file (e.g. wrong syntax).
- 3 Start Nginx and check the error.
- 4 Use logs to find the problem.
- 5 Fix the configuration.
- 6 Restart Nginx and confirm it works.

```
# Stop Nginx
$ sudo systemctl stop nginx

# Test the config (shows error)
$ sudo nginx -t

# Start Nginx (will fail)
$ sudo systemctl start nginx

# Check the logs
$ journalctl -u nginx -xe

# Fix the config, then restart
$ sudo systemctl restart nginx
```



JUNIOR ENGINEER TIP

When something is not working, check the logs first. The error message usually tells you exactly what went wrong and how to fix it.

6. COMMON LOG MESSAGES AND MEANINGS

Log Message	Meaning
Connection refused	Service is not running or not listening on the port.
Failed to bind to port	Port is already in use.
Permission denied	Wrong file permissions or user.
No such file or directory	Missing file or incorrect path.
Syntax error in config	Configuration file has an error.
Timeout	Network issue or service not reachable.

7. KEY TAKEAWAYS

- ✓ Logs are stored in /var/log and in the systemd journal.
- ✓ Use journalctl and tail -f to view logs.
- ✓ Always follow evidence instead of guessing.
- ✓ Logs help you find and fix real problems.
- ✓ Troubleshooting is a key skill for DevOps, Cloud and SRE roles.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Learn
Build
Improve
Repeat

Storage, Disks, and Filesystem Health

In this page, you will learn about disks, partitions, filesystems and mount points. You will use commands like lsblk, df -h, du -sh and mount, handle a disk-full incident and find what consumed the space.

Page 17 of 20

1. LINUX STORAGE OVERVIEW

Key components:

- Disks** Physical or virtual storage devices (e.g. /dev/sda).
- Partitions** Divides a disk into sections (e.g. /dev/sda1).
- Filesystems** How data is organized (e.g. ext4, xfs).
- Mount points** Directories where filesystems are attached (e.g. /, /home, /var).



Disk
→ Partition
→ Filesystem
→ Mount Point
→ Data

Everything in Linux is a file, and storage is accessed through the filesystem.

2. VIEW DISKS AND PARTITIONS (lsblk)

Use lsblk to list all disks, partitions and mount points.

```
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 50G 0 disk
├─sda1 8:1 0 1G 0 part /boot
├─sda2 8:2 0 49G 0 part /
└─sdb 8:16 0 100G 0 disk
  └─sdb1 8:17 0 100G 0 part /data
```

Shows your storage devices, partitions and where they are mounted.

TIP Use lsblk -f to also see the filesystem type (ext4, xfs, etc.).

3. CHECK FILESYSTEM USAGE (df -h)

Use df -h to see disk space usage.

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            3.9G     0  3.9G   0% /dev
tmpfs           794M   1.2M  792M   1% /run
/dev/sda2       49G    42G   4.1G  92% /
/dev/sda1       1.0G   320M  680M  33% /boot
/dev/sdb1      100G    10G   90G  10% /data
```

Shows total size, used space, available space and usage percentage for each mounted filesystem.

4. CHECK DIRECTORY SIZE (du -sh)

Use du -sh to find which directories are using the most space.

```
$ du -sh /* 2>/dev/null | sort -hr | head -n 10
18G  /var
12G  /home
4.1G /usr
2.3G /opt
1.1G /tmp
620M /boot
210M /etc
120M /var/log
56M  /root
32M  /srv
```

Shows the size of directories so you can find what is consuming space.

TIP Use sudo du -sh /var/* to check inside /var.

5. MOUNT AND UNMOUNT FILESYSTEMS

Use mount to view mounted filesystems.

```
$ mount | column -t
/dev/sda2 on / type ext4 (rw,relatime)
/dev/sda1 on /boot type ext4 (rw,relatime)
/dev/sda1 on /data type xfs (rw,relatime)
```

Shows all mounted filesystems and their options.

Common commands:

```
$ sudo mount /dev/sdb1 /data
$ sudo umount /data
$ sudo mkdir -p /data
```

Mount a filesystem
Unmount a filesystem
Create a mount point

6. DISK-FULL INCIDENT (TROUBLESHOOTING STEPS)

When a disk is full, follow these steps:

- 1 Check disk usage with `df -h`.
- 2 Find large directories with `du -sh / 2>/dev/null | sort -hr`.
- 3 Investigate the top directories (e.g. /var, /home, /docker).
- 4 Look for large log files, old backups, or unused files.
- 5 Remove or move unnecessary files (be careful).
- 6 Verify space is recovered with `df -h`.



A full disk can cause services to fail, applications to crash and the server to become unstable.

7. REAL EXAMPLE: FIND WHAT CONSUMED THE SPACE

Find the largest files in /var (common cause of disk full).

```
$ sudo find /var -type f -size +100M -exec ls -lh {} + | sort -k5 -hr | head
-rw-r--r-- 1 root root 2.3G Aug 26 10:12 /var/log/big-app.log
-rw-r--r-- 1 root root 1.1G Aug 25 09:30 /var/lib/docker/containers/.../log
-rw-r--r-- 1 root root 450M Aug 24 14:22 /var/log/nginx/access.log
```

Finds large files so you can safely remove or rotate them.

8. KEY TAKEAWAYS

- Understand disks, partitions, filesystems and mount points.
- Use lsblk, df -h, du -sh and mount confidently.
- Know how to handle a disk-full incident.
- Find what consumed the space using du and find.
- Regularly clean up logs and unnecessary files.
- Storage monitoring is essential for stable systems.



JUNIOR ENGINEER TIP

Set up log rotation (e.g. logrotate) and monitoring (e.g. using df alerts) to prevent disk-full incidents.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Bash and Linux Automation

In this page, you will learn the fundamentals of Bash scripting and automation in Linux. You will work with variables, exit codes, pipes and redirection, create simple scripts, automate system checks and schedule tasks with cron.

Page 18 of 20

1. VARIABLES

Variables store values that you can reuse in your commands and scripts.

```
# Define a variable
name="veriqta"
echo "Hello $name"
echo "Your home directory is $HOME"
echo "Current user: $USER"
```

Naming rules

- Use letters, numbers and underscore.
- No spaces (use _).
- Access with \$ before the name.
- Use quotes for values with spaces.



TIP Use double quotes (") to avoid issues with spaces.

2. EXIT CODES

Every command returns an exit code. 0 means success, non-zero means failure.

```
# Run a command
ls /etc
echo "Exit code: $?"
# Check success or failure
if [ $? -eq 0 ]; then
    echo "Command succeeded"
else
    echo "Command failed"
fi
```

Common exit codes

0	Success
1	General error
2	Misuse of shell command
126	Permission denied
127	Command not found



TIP Always check exit codes in scripts to handle errors properly.

3. PIPES AND REDIRECTION

Pipes and redirection help you work with command output.

```
# Pipe output to another command
ps aux | grep nginx
# Redirect output to a file
echo "System report" > report.txt
# Append output to a file
echo "New entry" >> report.txt
# Redirect errors
ls /notfound 2> error.log
# Redirect both output and errors
ls /notfound > output.log 2>&1
```

Common redirection

>	Overwrite file
>>	Append to file
2>	Redirect errors
2>&1	Redirect both output and errors
	Pipe to another command



Use pipes and redirection to filter, save and analyze output.

4. SIMPLE BASH SCRIPTS

Create a simple script to automate tasks.

```
#!/bin/bash
# system_info.sh
echo "=== System Information ==="
echo "Hostname: $(hostname)"
echo "Date: $(date)"
echo "Uptime: $(uptime)"
echo "Disk Usage:"
df -h
echo "Memory Usage:"
free -h
```

Make it executable

```
chmod +x system_info.sh
./system_info.sh
```

Best practices

- Use a shebang (#!/bin/bash)
- Add comments
- Use meaningful variable names
- Handle errors
- Make the script executable

5. AUTOMATE SYSTEM CHECKS

Create a script to check key system information.

```
#!/bin/bash
# check_system.sh
echo "=== System Check ==="
echo "Disk Usage:"
df -h | grep -E '^/dev/'
echo "\nMemory Usage"
free -h
echo "\nTop 5 Processes by CPU:="
ps aux --sort=-%cpu | head -n 6
echo "\nTop 5 Processes by Memory:="
ps aux --sort=-%mem | head -n 6
```

What this does

- Shows disk usage
- Shows memory usage
- Shows top CPU processes
- Shows top memory processes



You can expand this script to check services, disk space thresholds, or send alerts via email.

6. SCHEDULE A TASK WITH CRON

Use cron to run scripts automatically at specific times.

```
# Open crontab
crontab -e
# Run script every day at 8 AM
0 8 * * * /path/to/check_system.sh
# Run every 5 minutes
*/5 * * * * /path/to/check_system.sh
# View current cron jobs
crontab -l
# Remove all cron jobs
crontab -r
```

Cron time format

* * * * *	Day of week (0-7)
* * * * *	Month (1-12)
* * * * *	Day of month (1-31)
* * * * *	Hour (0-23)
* * * * *	Minute (0-59)



Examples:

```
0 2 * * * Run daily at 2 AM
0 * * * * Run every hour
*/15 * * * * Run every 15 minutes
```

7. KEY TAKEAWAYS

- ✓ Variables store and reuse values.
- ✓ Bash scripts automate repetitive tasks.
- ✓ Exit codes tell you if a command succeeded or failed.
- ✓ You can automate system checks with simple scripts.
- ✓ Pipes and redirection help you work with output.
- ✓ Cron lets you schedule tasks to run automatically.



JUNIOR ENGINEER TIP

Start small. Automate simple checks first, then gradually build more complex scripts. Automation saves time, reduces human error and makes you a more valuable engineer.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

VERIQTA

TECH KNOWLEDGE FOR A BRIGHTER TOMORROW

Learn
Build
Improve
Repeat

Secure and Operate Your Home Lab

In this page, you will learn how to keep your home lab secure, up to date and reliable. You will cover updates and patching, least privilege, SSH security, firewall review, backups, logs and disk monitoring, and what not to expose to the internet.

Page 19 of 20

1. UPDATES AND PATCHING

Keep your systems up to date to fix security vulnerabilities.

```
# Update package list
sudo apt update

# Upgrade packages
sudo apt upgrade -y

# Remove unused packages
sudo apt autoremove -y

# Check for security updates
sudo apt list --upgradable
```

Best practices

- ✓ Apply security updates regularly.
- ✓ Enable unattended upgrades (optional).
- ✓ Reboot if required.
- ✓ Monitor vendor security advisories.



Unpatched systems are one of the biggest security risks, even in a home lab.

2. LEAST PRIVILEGE

Give only the minimum access required.

```
# Create a standard user
sudo adduser devuser

# Add user to sudo group
sudo usermod -aG sudo devuser

# Switch to the new user
su - devuser
```

Principles

- ✓ Do not use root for daily tasks.
- ✓ Use `sudo` when needed.
- ✓ Give only the access required.
- ✓ Review user accounts regularly.



Least privilege reduces the damage if an account is compromised.

3. SSH SECURITY

Secure your SSH access.

```
# Disable password authentication
sudo nano /etc/ssh/sshd_config

# Set:
PasswordAuthentication no
PermitRootLogin no

# Restart SSH
sudo systemctl restart ssh
```

Best practices

- ✓ Use SSH keys instead of passwords.
- ✓ Disable root login.
- ✓ Change the default port (optional).
- ✓ Use fail2ban (optional).
- ✓ Keep your private keys safe.



Never use passwords or allow root login over SSH even in a home lab.

4. FIREWALL REVIEW

Allow only the services you need.

```
# Check firewall status (UFW)
sudo ufw status

# Allow only required services
sudo ufw allow ssh
sudo ufw allow 80 # if running a web server
sudo ufw allow 443 # if running HTTPS

# Deny everything else
sudo ufw default deny incoming
sudo ufw enable
```

Checklist

- ✓ Allow only necessary ports.
- ✓ Review rules regularly.
- ✓ Remove unused services.
- ✓ Do not expose internal services to the internet.



A firewall is your first line of defense. Keep it simple and tight.

5. BACKUPS

Back up your important data and configurations.

```
# Backup a directory
sudo tar -czf /backup/home_$(date +%F).tar.gz /home

# Backup important configs
sudo tar -czf /backup/etc_$(date +%F).tar.gz /etc

# (Optional) Copy backup to another server
scp /backup/*.tar.gz user@192.168.56.2:/backups/
```

What to back up

- Home directories
- Application data
- Configuration files
- Scripts and projects
- VM images (if possible)



Test your backups. A backup you cannot restore is useless.

6. LOGS AND DISK MONITORING

Monitor logs and disk usage to catch problems early.

```
# Check system logs
journalctl -xe

# Follow logs in real time
tail -f /var/log/syslog

# Check disk usage
df -h

# Find large directories
du -sh /* | sort -hr | head -n 10
```

Monitoring tips

- ✓ Check logs regularly for errors or unusual activity.
- Monitor disk usage.
- Set up alerts (e.g. email) for high disk usage.
- ✓ Clean up old logs and unnecessary files.

8. KEY TAKEAWAYS

- ✓ Keep your systems updated and patched.
- ✓ Use least privilege.
- ✓ Secure your SSH access.
- ✓ Review and configure your firewall.
- ✓ Back up your data and configuration.
- ✓ Monitor logs and disk usage.
- ✓ Avoid exposing unnecessary services to the internet.



JUNIOR ENGINEER TIP

A secure and well-maintained home lab helps you learn safely, prevents unnecessary problems and builds real-world habits.



@veriqta
Instagram



@veriqta
GitHub



@veriqta
LinkedIn



@veriqta
Telegram



@veriqta
Medium



@veriqta
X

Final Project: Run Your Own Mini Production Environment

In this final project, you will set up two Linux servers and run a real, working environment. You will configure users, SSH, Nginx, firewall rules, monitoring and logging. You will intentionally break Nginx, diagnose the issue, fix it and verify recovery. This project brings together everything you have learned.

Page 20 of 20

1. ENVIRONMENT OVERVIEW

You will use two Linux servers (VMs).

- Server 1: Web server (Nginx)
- Server 2: Internal server (for networking, service-to-service communication and troubleshooting)
- Both servers on the same private network
- No root login (use a dedicated admin user)
- SSH key authentication
- Proper firewall rules
- Monitoring, logs and a health-check script

2. CREATE SERVERS AND USERS

Suggested setup:

Server	Hostname	Private IP	Role
Server 1	web01	192.168.56.101	Nginx web server
Server 2	app01	192.168.56.102	Internal server

```
# On both servers
sudo adduser devops
sudo usermod -sG sudo devops
# Disable root login (optional but recommended)
sudo nano /etc/ssh/sshd_config
# PermitRootLogin no
sudo systemctl restart ssh
```

3. SSH KEY AUTHENTICATION

Generate keys on your local machine and copy to both servers.

```
# On local machine
ssh-keygen -t ed25519 -C "veriqta"
# Copy to servers
ssh-copy-id devops@192.168.56.101
ssh-copy-id devops@192.168.56.102
```



TIP

Keep your private key safe and set correct permissions.

4. INSTALL AND CONFIGURE NGINX (SERVER 1)

Install Nginx and make it run automatically.

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx
```

Verify:

```
curl http://localhost
curl http://192.168.56.101
```

5. FIREWALL RULES

Allow only required services.

```
sudo ufw default deny incoming
sudo ufw allow ssh
sudo ufw allow 80
sudo ufw allow 443
sudo ufw enable
sudo ufw status
```

6. SERVER-TO-SERVER COMMUNICATION

Test private network connectivity.

```
# From server 2 to server 1
ping 192.168.56.101
ssh devops@192.168.56.101
curl http://192.168.56.101
```



Both servers should be able to communicate over the private network.

7. BASH HEALTH-CHECK SCRIPT

Create a script to check Nginx and system health.

```
#!/bin/bash
# health_check.sh
if systemctl is-active --quiet nginx; then
    echo "Nginx is running"
else
    echo "Nginx is NOT running"
fi
df -h | grep -E '^/dev/'
free -h
uptime
```

Make it executable:

```
chmod +x
health_check.sh
```

Run it:

```
./health_check.sh
```

8. LOGS AND SERVICE MONITORING

Use logs to monitor and troubleshoot.

```
# Nginx logs
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
# System logs
journalctl -u nginx -f
# Check service status
systemctl status nginx
```

9. INTENTIONALLY BREAK NGINX

Stop Nginx or use a bad configuration.

```
# Option 1: Stop the service
sudo systemctl stop nginx
# Option 2: Break the config
sudo nano /etc/nginx/nginx.conf
# Add an invalid line, save and test
sudo nginx -t
```

10. DIAGNOSE THE FAILURE

Use logs and commands to find the issue.

- Check service status
- Read the error logs
- Identify the root cause
- Verify configuration with `nginx -t`

11. REPAIR AND VERIFY RECOVERY

Fix the issue and confirm Nginx is working again.

- Fix the configuration or start the service
- Test with `curl`
- Check logs for errors
- Make sure the service is stable
- Confirm it starts automatically after reboot

12. FINAL ARCHITECTURE

Your environment will look like this:

